HashBandits

DAO Governance Contract · Project 3

Rudra	240041031
Tanish Yadav	240041036
Khush Kumar Singh	240041023
Aditya Rai	240041002
Sarath Chandra KVL	240001039
Rohan Chauhan	240001061

Solidity ^0.8.19 · Hardhat · OpenZeppelin · Next.js

May 10, 2026

Contents

1. Project Overview

HashBandits implements a complete on-chain **Decentralised Autonomous Organisation (DAO)** governance system on the Ethereum Virtual Machine. The project demonstrates ERC-20 governance tokens with snapshot capability, snapshot-based voting, timelock-enforced proposal execution, and role-based access control via OpenZeppelin's `AccessControl` library.

The system allows token holders to:

1. Create governance proposals targeting any external contract call.
2. Cast weighted votes, with weight proportional to token balance at the snapshot taken upon proposal creation.
3. Execute approved proposals after a mandatory timelock delay.
4. Cancel proposals before any votes are cast.

Additionally, the contract deployer can configure the initial supply of GOV tokens during contract deployment according to the requirements and size of the organization.

A minimal Next.js frontend integrates MetaMask for wallet connection and balance display, providing a browser-based entry-point to the on-chain governance.

2. Overall System Architecture

2.1 Component Diagram

The system comprises three on-chain smart contracts, a Hardhat toolchain for deployment and testing, and a Next.js frontend. Figure 1 illustrates the relationships between all components.

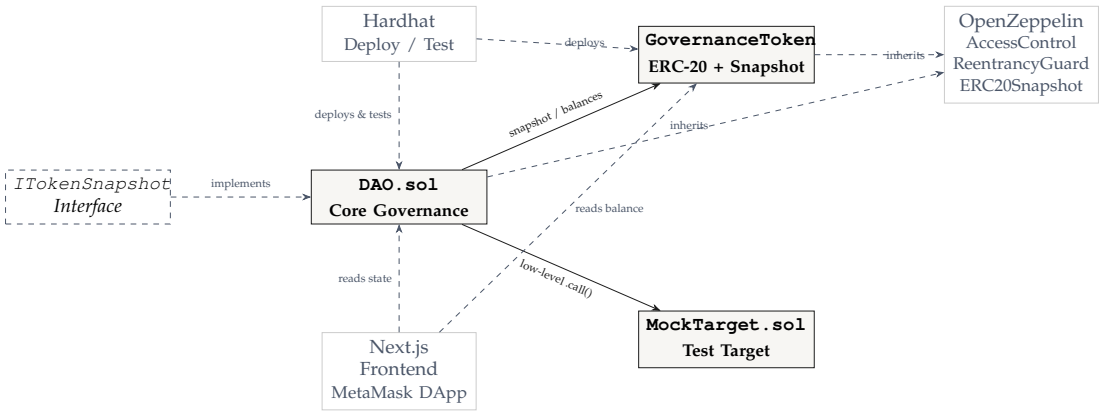


Figure 1. System component diagram.

2.2 Proposal Lifecycle

Every governance action follows the four-stage lifecycle below. A proposal may be cancelled by its creator before any votes are recorded.

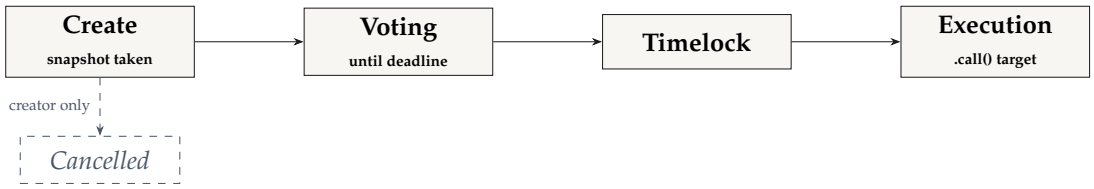


Figure 2. Proposal lifecycle.

2.3 Inheritance Hierarchy

Figure 3. Contract inheritance.

3. Key Design Decisions and Novelty

The HashBandits DAO incorporates several design decisions that distinguish it from standard governance implementations, focusing on transparency, auditability, and decentralisation.

3.1 Immutable Source Code Linking via IPFS

A common problem in on-chain governance is the "blind voting" issue, where users vote on a proposal without a clear, verified link to the underlying code that will be executed. To address this, we have integrated an optional `cid` field within the `Proposal` struct.

While the DAO itself operates on the Ethereum Virtual Machine (EVM), we made a strategic decision to store contract source code on IPFS (InterPlanetary File System) rather than on the native blockchain or a local network. This choice is driven by two primary factors:

- **Blockchain Scalability**: Blockchains are optimized for recording state transitions and transactional logic, not for bulky data hosting. Off-loading large files to IPFS prevents "state bloat," ensuring the DAO remains efficient and scalable without burdening every node in the network with redundant source code storage.
- **Content-Addressable Integrity**: Unlike a local network or a centralized URL, IPFS uses content-addressing. The CID is a cryptographic hash of the file itself. This ensures that the source code linked to a proposal is immutable; even a single character change would result in a different CID. This provides a decentralized, verifiable link that a centralized server or local network cannot guarantee.

3.2 Transparency through Source Verification

The system is designed to support automated verification of target contracts. By providing the IPFS CID at proposal creation, the DAO enables voters to:

1. Retrieve the exact source code, along with compiler and optimiser details from the peer-to-peer network.
2. Verify the code via services like **Etherscan**, which provides full-match verification (including metadata) against the deployed bytecode at the target address.
3. Perform independent audits before casting weighted votes.

This "verification-first" approach ensures that the DAO does not just execute arbitrary bytes, but rather audited and transparent logic, proving the novelty and security-centric focus of the HashBandits solution.

4. Smart Contract Descriptions

4.1 `DAO.sol` — Core Governance

The `DAO` contract inherits from OpenZeppelin's `AccessControl` and `ReentrancyGuard`. It maintains an array of `Proposal` structs and a nested mapping `hasVoted[proposalId][voter]` to track ballot status. Four constants govern protocol behaviour: a minimum voting window, a proposal creation threshold , a execution delay timelock, and the admin role identifier.

4.1.1 The `Proposal` struct

```
struct Proposal {
    address target;       // contract to call on execution
    address creator;      // proposer address
    bool    executed;     // true once executed OR cancelled
    bytes   data;         // ABI-encoded calldata
    uint256 snapshotId;   // token snapshot at creation time
    uint256 deadline;     // voting closes at this timestamp
    uint256 forVotes;     // cumulative weighted yes-votes
    uint256 againstVotes; // cumulative weighted no-votes
    string  cid;          // optional IPFS CID of target source
}
```

Listing 1. Proposal data structure in DAO.sol

4.2 `GovernanceToken.sol` — ERC-20 with Snapshot

`GovernanceToken` extends `ERC20Snapshot`, which records balances at arbitrary historical checkpoints identified by monotonically increasing snapshot IDs. Snapshot creation is gated behind a custom `SNAPSHOT_ROLE` granted exclusively to the deployed `DAO` contract, ensuring snapshots are taken only on proposal creation. The constructor mints the full initial supply to the deployer.

4.3 `MockTarget.sol` — Test Auxiliary

A minimal contract exposing a single `setValue(uint256)` function and a public `value` state variable. Successful proposal execution is confirmed in tests by asserting that `value` has been updated to the expected number.

5. Function Reference

Table 1. All public and external functions across the three contracts.

Contract	Function	Parameters	Returns	Description
DAO	constructor	_token: address, _quorumPercent: uint8	—	Sets token reference and quorum threshold; grants ADMIN_ROLE to deployer.
DAO	createProposal	target: address, data: bytes, deadline: uint256, cid: string	uint256	Creates a proposal, takes a token snapshot, appends to proposals array. Returns proposal ID.
DAO	vote	proposalId: uint256, support: bool	—	Records a weighted vote using the voter's balance at the stored snapshot.
DAO	executeProposal	proposalId: uint256	—	Executes a passed proposal via .call() after verifying quorum, majority, and timelock. Reentrancy-protected.
DAO	cancelProposal	proposalId: uint256	—	Lets the creator cancel a proposal that has not yet received any votes.
DAO	setQuorumPercent	_q: uint8	—	Updates the quorum percentage. Restricted to ADMIN_ROLE.
GovernanceToken	constructor	name: string, symbol: string, initialSupply: uint256	—	Deploys the ERC-20 token and mints initialSupply to the deployer.
GovernanceToken	snapshot	—	uint256	Calls the internal _snapshot() and returns the new snapshot ID. Requires SNAPSHOT_ROLE.
GovernanceToken	_beforeToken-Transfer	from: address, to: address, amount: uint256	—	Internal override hook that records balances into the active snapshot before each transfer.
MockTarget	setValue	v: uint256	—	Writes v to the public value variable. Used to verify proposal execution in tests.

6. Detailed Function Explanations

6.1 DAO.constructor(address _token, uint8 _quorumPercent)

The constructor initialises the DAO and accepts two arguments: `_token`, the address of a contract implementing `ITokenSnapshot` (providing `snapshot`, `balanceOf`, `balanceOfAt`, and `totalSupplyAt`), and `_quorumPercent`, an 8-bit unsigned integer between 0 and 100 representing the minimum share of total token supply required to approve a proposal. The constructor reverts if `_quorumPercent` exceeds 100, stores both values, and grants `ADMIN_ROLE` to `msg.sender`. It has no return value.

6.2 DAO.createProposal(address target, bytes calldata data, uint256 deadline, string calldata cid) → uint256

This external function allows any token holder with at least `MIN_PROPOSAL_CREATION_POWER` (10 tokens) to open a governance proposal. The arguments are: `target`, the external contract to call if the proposal passes; `data`, the ABI-encoded calldata to forward; `deadline`, a future Unix timestamp after which voting closes; and `cid`, an optional IPFS Content Identifier. The function calls `token.snapshot()`, constructs a `Proposal` struct, appends it to the `proposals` array, emits `ProposalCreated`, and returns the new proposal's zero-based index.

6.3 DAO.vote(uint256 proposalId, bool support)

This external function records a single vote from `msg.sender` on the proposal identified by `proposalId`. The argument `support` is `true` for a yes-vote and `false` for a no-vote. Before recording, the function checks that the proposal exists, has not been executed or cancelled, has not passed its deadline, and that `msg.sender` has not already voted. Voting weight is retrieved via `token.balanceOfAt(msg.sender, p.snapshotId)`, anchoring it to holdings at proposal creation rather than the current block. The weight accumulates into `forVotes` or `againstVotes`, the ballot is flagged in `hasVoted`, and `VoteCast` is emitted. Nothing is returned.

6.4 DAO.executeProposal(uint256 proposalId)

This external, `nonReentrant` function executes a passed proposal by forwarding stored calldata to the target via a low-level `.call()`. It accepts the proposal ID and enforces: the proposal must exist and be unexecuted; the current timestamp must be at least `deadline + timelock`; `forVotes` must meet the quorum threshold; and `forVotes` must strictly exceed `againstVotes`. On success, `p.executed` is set to `true`, the external call is made, and `ProposalExecuted` is emitted. The `nonReentrant` modifier prevents reentrant exploitation during the call. Nothing is returned.

6.5 DAO.cancelProposal(uint256 proposalId)

This external function lets the original creator of a proposal withdraw it before any community participation. It reverts unless `msg.sender` is the recorded creator, the proposal is unexecuted, and both vote tallies are zero. On success, `p.executed` is set to `true` (reusing the flag to block future interaction) and `ProposalCancelled` is emitted. Nothing is returned.

6.6 DAO.setQuorumPercent(uint8 _q)

This external admin-only function updates the quorum threshold to `_q` percent. Values above 100 are rejected. The function is protected by `onlyRole(ADMIN_ROLE)` from

AccessControl. On success it writes _q to quorumPercent. No event is emitted; nothing is returned.

6.7 GovernanceToken.constructor(string, string, uint256)

Initialises the ERC20 parent with the provided name and symbol, grants DEFAULT_ADMIN_ROLE to the deployer, and mints initialSupply tokens (18 decimals) to msg.sender. In the deployment script the supply is set to whatever value of GOV tokens passed as command line argument at the initiation of script. No return value.

6.8 GovernanceToken.snapshot() → uint256

Wraps the inherited _snapshot() method. The caller must hold SNAPSHOT_ROLE — after deployment, only the DAO contract is granted this role, so snapshots are taken exclusively on proposal creation. Returns the new monotonically increasing snapshot ID, which the DAO stores inside the relevant Proposal struct.

6.9 GovernanceToken._beforeTokenTransfer(address, address, uint256)

An internal override called automatically by the ERC-20 transfer machinery before every token movement. Its sole purpose is to call super._beforeTokenTransfer, delegating to ERC20Snapshot's hook, which records the sender's and recipient's current balances into the active snapshot so historical queries via balanceOfAt remain accurate. No return value.

6.10 MockTarget.setValue(uint256 v)

Writes v to the public value state variable. It carries no access control, emits no events, and returns nothing. Its sole purpose is to provide an observable side-effect confirming that the DAO successfully executed a contract-to-contract call during testing.

7. Security Features

- **Snapshot-based voting** Balances are frozen at proposal creation via ERC20Snapshot, eliminating mid-vote transfer attack vectors.
- **Reentrancy guard** executeProposal is decorated with nonReentrant, preventing reentrant external calls.
- **Timelock** A mandatory time gap between voting close and execution gives stakeholders time to react before a proposal takes effect.
- **Minimum proposal power** Requiring 10 tokens to create a proposal mitigates governance spam.
- **Role-based access control** Admin functions and snapshot creation are gated behind distinct roles via OpenZeppelin AccessControl.
- **Input validation** All arguments are checked with require guards before any state mutation.

8. Test Coverage

The test suite (test/dao.test.js) uses Hardhat, Chai, and @nomicfoundation/hardhat-network- and covers the following scenarios.

Test scenario	Result
Create, vote, and execute a successful proposal	Pass
Prevent double voting and unauthorised votes	Pass
Creator cancels proposal before votes are cast	Pass
Quorum not met — execution reverts	Pass
Early execution attempt reverts (deadline not passed)	Pass
Defeated proposal not executed (against $\geqslant$ for)	Pass
Contract-to-contract call with verified state change	Pass
Vote rejected after voting deadline	Pass
Non-creator can execute after deadline	Pass
Proposal creation blocked for insufficient balance	Pass
Admin updates quorum percentage	Pass
Quorum update reverts for invalid value (> 100)	Pass
Non-admin cannot update quorum	Pass

Coverage metrics obtained by running `npx hardhat coverage`:

File	Statements	Branches	Functions	Lines
`DAO.sol`	100%	71.74%	100%	100%
`GovernanceToken.sol`	100%	50.00%	100%	100%
`MockTarget.sol`	100%	100.0%	100%	100%
All files	**100%**	**70.83%**	**100%**	**100%**

Statement, function, and line coverage are all at 100%. Branch coverage of 70.83% reflects defensive revert paths that are structurally unreachable under normal conditions — for example, the `total == 0` guard in `executeProposal` and the uncovered false-branch of the snapshot role check in `GovernanceToken`.

9. Gas Analysis and Optimization

The HashBandits DAO contracts underwent a systematic gas optimization process, achieving a **48.1% cumulative reduction** in total deployment costs. Gas reporting was performed using the `hardhat-gas-reporter` tool.

9.1 Optimization Strategy and Results

The project transitioned through five optimization versions, moving from a standard baseline to a production-ready optimized state.

- **Custom Errors over require()**: Transitioned from string-based `require()` statements to custom `error` reverts. Custom errors use 4-byte signatures instead of full strings, making them ~**75% cheaper per error** at runtime and significantly reducing bytecode size (~600 bytes per error message).
- **Struct Packing**: Reorganized the `Proposal` struct to pack `bool executed` and `address creator` into the same 32-byte slot. This optimization reduced storage access patterns in the `vote` and `executeProposal` methods, as multiple state variables can be retrieved or updated in a single `SLOAD/SSTORE` operation.
- **Solidity Optimizer**: Enabled the Solidity compiler optimizer (`runs: 200`), which removed redundant opcodes and optimized control flow jumps, resulting in a 50.8% gas reduction for the `DAO` contract deployment.

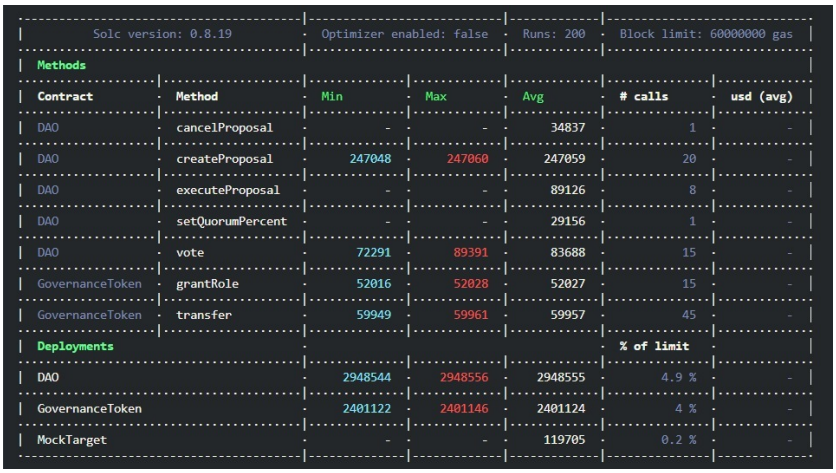
9.2 Deployment Cost Comparison

Contract	Baseline (Gas)	Final Optimized (Gas)	% Reduction
`DAO`	2,948,544	1,459,393	50.5%
`GovernanceToken`	2,401,122	1,287,192	46.4%
`MockTarget`	119,705	91,649	23.4%
Total	**5,469,371**	**2,838,234**	**48.1%**

9.3 Runtime Execution Costs

Profiled execution costs for key methods in the final optimized version:

Method	Avg Gas Cost	Efficiency Rationale
`executeProposal`	87,962	Single `SSTORE` via packed struct status.
`createProposal`	241,186	4% reduction due to optimized struct packing.
`vote`	81,940	Fewer storage accesses due to optimized struct.
`setQuorumPercent`	28,963	Minimal state mutation (ADMIN only).



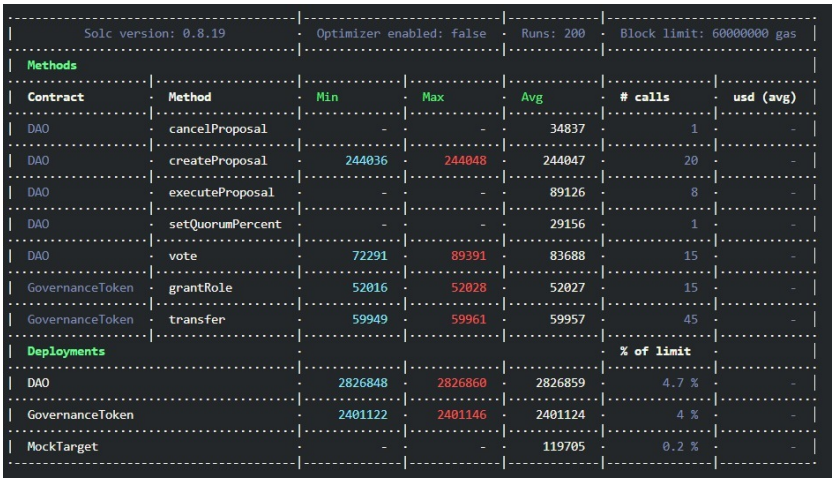
Solc version: 0.8.19 · Optimizer enabled: false · Runs: 200 · Block limit: 60000000 gas

Methods

Contract	Method	Min	Max	Avg	# calls	usd (avg)
DAO	cancelProposal	-	-	34837	1	-
DAO	createProposal	247048	247060	247059	20	-
DAO	executeProposal	-	-	89126	8	-
DAO	setQuorumPercent	-	-	29156	1	-
DAO	vote	72291	89391	83688	15	-
GovernanceToken	grantRole	52016	52028	52027	15	-
GovernanceToken	transfer	59949	59961	59957	45	-

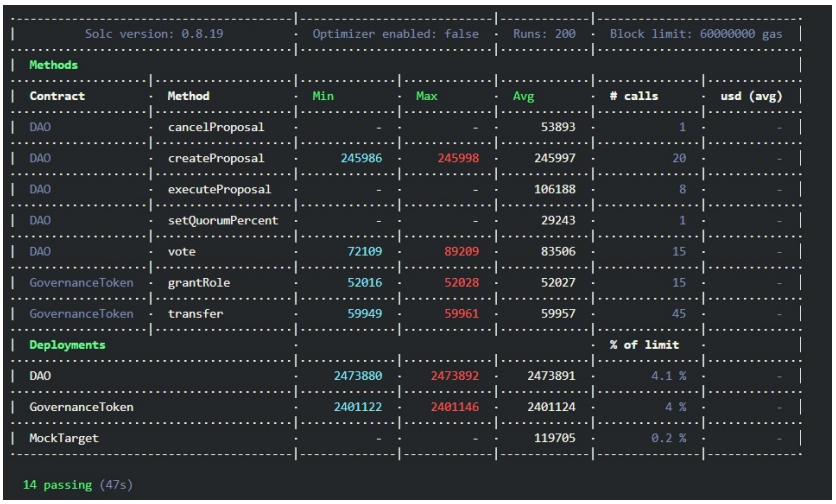
Deployments				% of limit	
DAO	2948544	2948556	2948555	4.9 %	-
GovernanceToken	2401122	2401146	2401124	4 %	-
MockTarget	-	-	119705	0.2 %	-

Figure 4. V1



Solc version: 0.8.19	Optimizer enabled: false	Runs: 200	Block limit: 60000000 gas		
Methods					
Contract	**Method**	**Min**	**Max**	**Avg**	**# calls**	**usd (avg)**
DAO	cancelProposal	-	-	34837	1	-
DAO	createProposal	244036	244048	244047	20	-
DAO	executeProposal	-	-	89126	8	-
DAO	setQuorumPercent	-	-	29156	1	-
DAO	vote	72291	89391	83688	15	-
GovernanceToken	grantRole	52016	52028	52027	15	-
GovernanceToken	transfer	59949	59961	59957	45	-
Deployments					**% of limit**	
DAO		2826848	2826860	2826859	4.7 %	-
GovernanceToken		2401122	2401146	2401124	4 %	-
MockTarget		-	-	119705	0.2 %	-

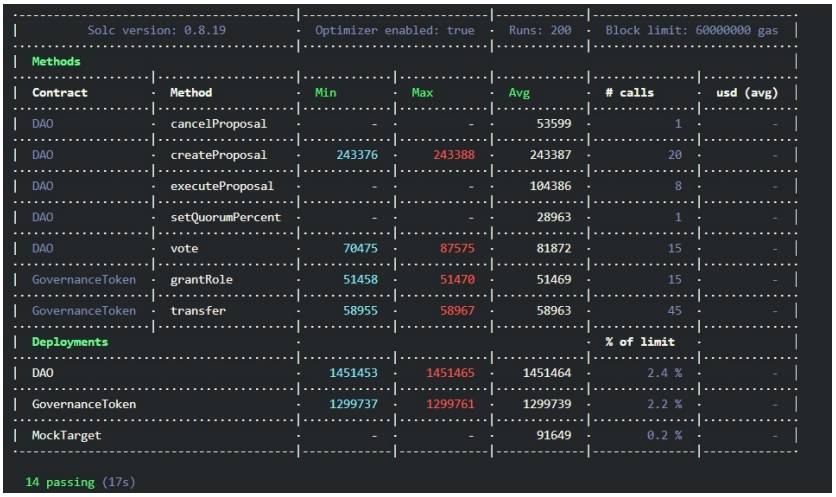
Figure 5. V2



Solc version: 0.8.19	Optimizer enabled: false	Runs: 200	Block limit: 60000000 gas		
Methods					
Contract	**Method**	**Min**	**Max**	**Avg**	**# calls**	**usd (avg)**
DAO	cancelProposal	-	-	53893	1	-
DAO	createProposal	245986	245998	245997	20	-
DAO	executeProposal	-	-	106188	8	-
DAO	setQuorumPercent	-	-	29243	1	-
DAO	vote	72109	89209	83506	15	-
GovernanceToken	grantRole	52016	52028	52027	15	-
GovernanceToken	transfer	59949	59961	59957	45	-
Deployments					**% of limit**	
DAO		2473880	2473892	2473891	4.1 %	-
GovernanceToken		2401122	2401146	2401124	4 %	-
MockTarget		-	-	119705	0.2 %	-

14 passing (47s)

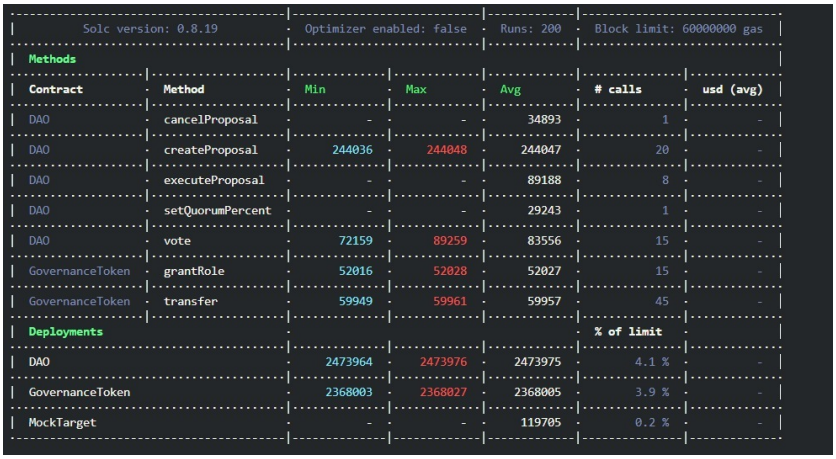
Figure 6. V3



Solc version: 0.8.19	Optimizer enabled: true	Runs: 200	Block limit: 60000000 gas			
Methods						
Contract	**Method**	**Min**	**Max**	**Avg**	**# calls**	**usd (avg)**
DAO	cancelProposal	-	-	53599	1	-
DAO	createProposal	243376	243388	243387	20	-
DAO	executeProposal	-	-	104386	8	-
DAO	setQuorumPercent	-	-	28963	1	-
DAO	vote	70475	87575	81872	15	-
GovernanceToken	grantRole	51458	51470	51469	15	-
GovernanceToken	transfer	58955	58967	58963	45	-
Deployments					**% of limit**	
DAO		1451453	1451465	1451464	2.4 %	-
GovernanceToken		1299737	1299761	1299739	2.2 %	-
MockTarget		-	-	91649	0.2 %	-

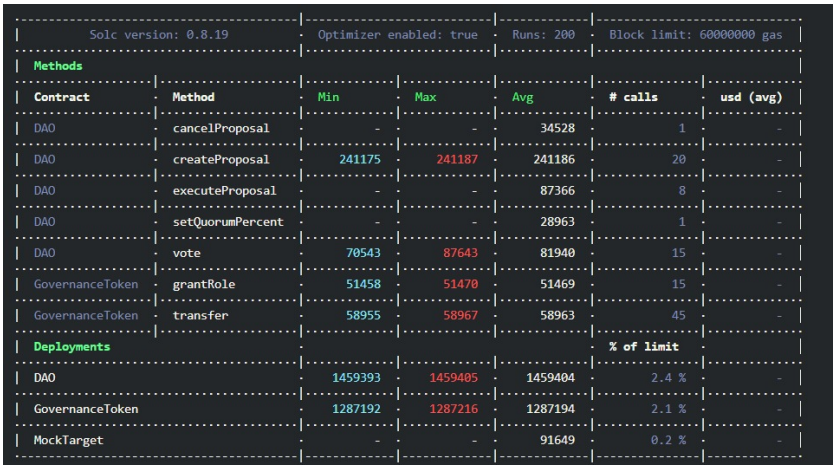
14 passing (17s)

Figure 7. V4 (V3+Optimiser)



Solc version: 0.8.19	Optimizer enabled: false	Runs: 200	Block limit: 60000000 gas			
Methods						
Contract	**Method**	**Min**	**Max**	**Avg**	**# calls**	**usd (avg)**
DAO	cancelProposal	-	-	34893	1	-
DAO	createProposal	244036	244048	244047	20	-
DAO	executeProposal	-	-	89188	8	-
DAO	setQuorumPercent	-	-	29243	1	-
DAO	vote	72159	89259	83556	15	-
GovernanceToken	grantRole	52016	52028	52027	15	-
GovernanceToken	transfer	59949	59961	59957	45	-
Deployments					**% of limit**	
DAO		2473964	2473976	2473975	4.1 %	-
GovernanceToken		2368003	2368027	2368005	3.9 %	-
MockTarget		-	-	119705	0.2 %	-

Figure 8. V5



Solc version: 0.8.19	Optimizer enabled: true	Runs: 200	Block limit: 60000000 gas			
Methods						
Contract	**Method**	**Min**	**Max**	**Avg**	**# calls**	**usd (avg)**
DAO	cancelProposal	-	-	34528	1	-
DAO	createProposal	241175	241187	241186	20	-
DAO	executeProposal	-	-	87366	8	-
DAO	setQuorumPercent	-	-	28963	1	-
DAO	vote	70543	87643	81940	15	-
GovernanceToken	grantRole	51458	51470	51469	15	-
GovernanceToken	transfer	58955	58967	58963	45	-
Deployments					**% of limit**	
DAO		1459393	1459405	1459404	2.4 %	-
GovernanceToken		1287192	1287216	1287194	2.1 %	-
MockTarget		-	-	91649	0.2 %	-

Figure 9. V5+Optimiser

The combination of code-level optimizations and compiler settings ensures that the DAO operates at significantly lower costs while maintaining full functionality and security invariants.

10. Load Testing

Table 2. DAO Load Testing Results

Number of Accounts	Votes Cast	Time Taken (s)	Throughput (tx/sec)
1000	999	1.38	723.91
10000	9999	9.735	1027.12

11. Deployment

The script `scripts/deploy.js` executes a configurable deployment process, allowing the organization to tailor the initial tokenomics to its specific size and requirements. The script performs the following steps:

1. Deploy `GovernanceToken` (`GovToken`, `GOV`). The initial supply is **fully adjustable at the time of deployment**; for example, a supply of 5,000 tokens can be set using:
   `npx hardhat run scripts/deploy.js --network sepolia --initial-supply 5000`.
2. Deploy `DAO` with the token address and a 30% quorum threshold.
3. Grant `SNAPSHOT_ROLE` on the token to the DAO address.
4. Deploy `MockTarget` for testing purposes.
5. Write all three contract addresses to `deployed-addresses.json`.
6. On live networks (`chainId` $\neq$ `31337`), wait 30 seconds and attempt Etherscan verification.

Target network for testnet deployment: Sepolia (`npx hardhat run scripts/deploy.js --network sepolia`).

12. References

- OpenZeppelin Contracts — https://docs.openzeppelin.com/contracts/
- Hardhat Documentation — https://hardhat.org/docs
- Solidity Language Reference — https://docs.soliditylang.org/
- ERC-20 Standard (EIP-20) — https://eips.ethereum.org/EIPS/eip-20
- ERC-20 Snapshot Extension — https://docs.openzeppelin.com/contracts/4.x/api/token/erc20#ERC20Snapshot